FP 11 (Haskell 7): Point-free programming

Cael Marquard
mrqcae001@myuct.ac.za
(slides adapted from Dr Sabre Didi)

Aim of this lecture

By the end of this lecture you should:

- Know how to identify point-free programming
- Be able to write point-free expressions of one argument

Many styles, one output

By now, you've probably noticed that Haskell often has **multiple ways of writing the same thing.** We could write the `add'` function in all these ways:

```haskell
add' :: Num a => a -> a -> a
add' a + b = a + b

-- With elision of type signature
add' a + b = a + b

-- As function returning lambda
add' a = \b -> a + b

-- Using where clause
add' a b = res
    where
  res = a + b
```

```haskell
-- Using let-in clause
-- These are actually expressions, so
can be used anywhere!
add' a b = let res = a + b
           in res

-- In Polish notation
add' a b = (+) a b

-- With no arguments specified
add' = (+)
```

Many styles, one output (2)

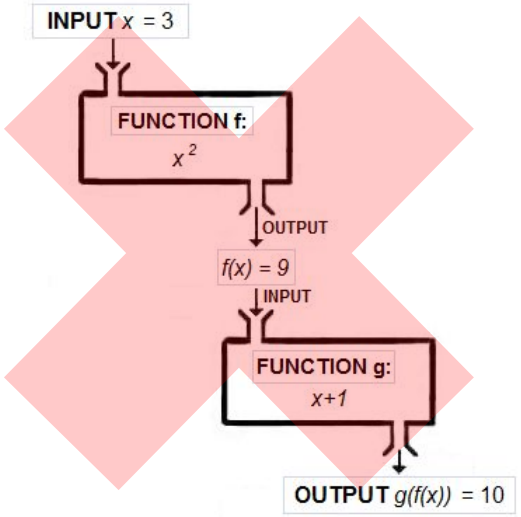
We could also write the expression `add' 1 (2 - 3)` either like that, or as `add' 1 $ 2 - 3`, using `$` to remove the grouping brackets.

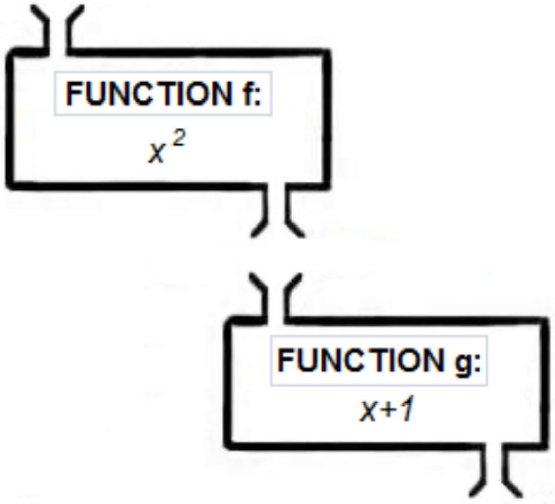
Once again, **it's often a matter of taste and context.** Sometimes, it is more natural to express a function in one way than another.

In this lecture, we will discuss **thinking about functions as compositions of other functions.**

Function composition

We sometimes think about functions (esp. in maths) as little **machines** which transform input values into output values. But, Haskell encourages us to try to think of the **functions themselves** rather than the values going in and out, through function composition.





Functions as composing other functions

Let's do an example with our best friend—prime number search!

We had previously defined isPrime in a few ways. Let's take a look at one sensible definition:

```
isPrime n = all (\f -> (n `mod` f) /= 0) [2..(n - 1)]
```

We have a lambda expression here in the middle operating on each potential factor, `f`, and returning if it is not a factor of `n`. We can rewrite this as a **composition of functions** like this:

```
isPrime n = all ((/= 0) . mod n) [2..(n - 1)]
```

Functions as composing other functions (2)

Now let's look at kthNextPrime as a composition of functions. Previously, we defined it recursively as follows:

```
kthNextPrime :: Int -> Int -> Int
kthNextPrime start k = searchPrime (start + 1) (k - 1)
  where
    searchPrime start k
      | isPrime start && k == 0 = start
      | isPrime start = searchPrime (start + 1) (k - 1)
      | otherwise = searchPrime (start + 1) k
```

Let's rather try define this in terms of generating and manipulating a *sequence of primes.*

Functions as composing other functions (2)

Let's generate an infinite sequence of primes after `start`. Remember that, because Haskell is lazy, **we won't evaluate any more than we need.**

```
filter isPrime [start..]
```

Now we need to get the kth element after start, so we can simply write

```
kthNextPrime start k = filter isPrime [start..] !! k
```

But, notice that we can **elide the argument k:**

```
kthNextPrime start   = (filter isPrime [start..] !!)
```

Functions as composing other functions (3)

```
kthNextPrime start = (filter isPrime [start..] !!)
```

Bear with me while I reorder !! into prefix/Polish notation, rather than infix notation:

```
kthNextPrime start =  (!!) (filter isPrime [start..])
```

Now that we've done this, we can elide the argument **start** using the `enumFrom` function to replace our generator literal:

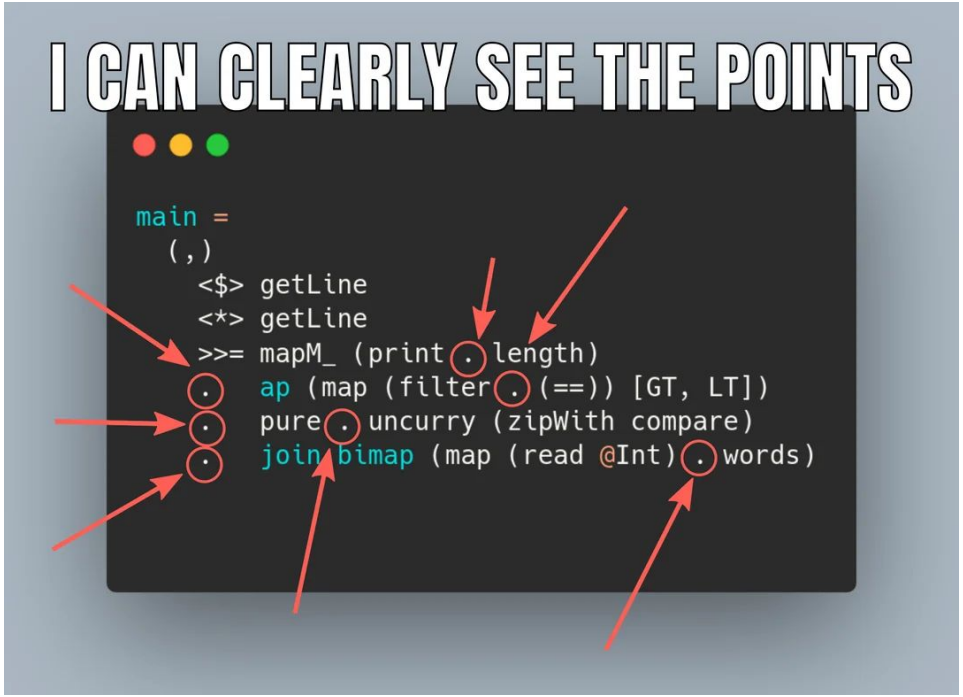
```
kthNextPrime start = (!!) (filter isPrime (enumFrom start))
kthNextPrime       = (!!) . filter isPrime . enumFrom
```

Our function now **does not even refer** to its arguments! This is called *point-free programming.*

Point-free programming

Point-free programming doesn't mean that there are no points (dots), OR that you are programming something pointless (har-har)...

… it is when you write functions that **do not name their arguments** (the "**points**" that this style is free from)




```
kthNextPrime start k = filter isPrime [start..] !! k

kthNextPrime = (!!) . filter isPrime . enumFrom
```

"Pointful"

Point-free

Point-free programming - advantages & disadvantages

Advantage 1: point-free programming can be clearer than pointful programming. This is because you sometimes introduce variables for no reason other than to pass them straight to a function.

For instance, it is much clearer to write `foldl' (+) 0 xs` ("we are folding `xs` with +") than to write the pointful `foldl' (\acc x -> acc + x) xs` ("we are adding acc and x together for each element x of xs"). **Point-free expresses our intent better here!**

Disadvantage 2: Sometimes, the opposite is true. Personally, I prefer our pointful `kthNextPrime`! It depends on the context and is also up to your taste.

```
kthNextPrime start k = filter isPrime [start..] !! k

kthNextPrime = (!!) . filter isPrime . enumFrom
```

"Pointful"

Point-free

Point-free programming - advantages & disadvantages (2)

Advantage 2: point-free programming makes us think at a **higher, declarative level**: about what our program *is*, about composing functions. Not how we choose to shuffle data around.

```
vectorMagnitude :: [Float] -> Float
vectorMagnitude = sqrt . sum . map (^2)
```

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

Disadvantage 2: it becomes hard to follow if we have to **use the same argument in multiple places.** E.g. compare

```
isPrime n = all ((/= 0) . mod n) [2..(n - 1)]
isPrime = ap (all . ((0 /=) .) . mod) (enumFromTo 2 . subtract 1)
```

Point-free programming - advantages & disadvantages (3)

Advantage 3: point-free programming is sometimes considered as the **purest expression of functional programming,** since **everything is functions, all the way down.**

Disadvantage 3: it can be difficult to follow what the **types are** in a long chain of point-free code, since we have less clues in variable bindings.

For example:

```
example = (. (7.0 +)) . (/) . fromIntegral . (10 +)
```

… what types are the arguments supposed to be?

(it's Int -> Float -> Float)

Exercises

(please remember that if you use AI to do the following, it no longer has any value. You are only cheating yourself.)

Composition / point-free

- Implement `sum` in terms of `foldl'`, in point-free style
- Implement `isEven :: Int -> Bool` in point-free style